

Robótica
Informe capítulo 5– RVC v2 Peter Corke

Navegación autónoma
Parte 2
Celdas de ocupación y hojas de ruta

Docente: Jaime Enrique Arango Castro

Estudiante:
Cristhian Mateo Almeida Gómez

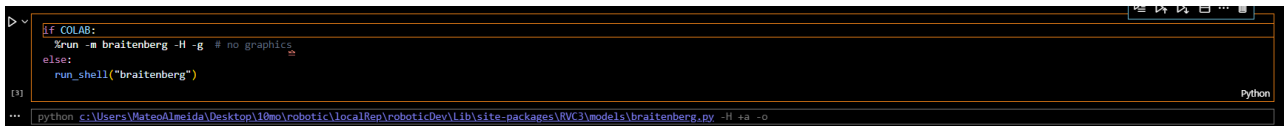
Universidad Nacional de Colombia
Sede Manizales



Resumen

Las secciones 5.2, 5.4 y 5.5 del capítulo 5 de *Robotics, Vision and Control* de Peter Corke introduce los fundamentos de la navegación robótica, enfocándonos en la navegación basada en hojas de ruta y celdas de ocupación, procedemos viendo que, la navegación basada en mapas tiene como objetivo determinar el mejor camino entre dos puntos en un entorno, utilizando información espacial predefinida, como mapas. A continuación, se describen las representaciones de mapas utilizadas en este tipo de navegación y los métodos aplicados.

Para llevar a cabo las simulaciones correspondientes a estos métodos, se utiliza la herramienta **BDSim**, un entorno de simulación basado en diagramas de bloques, que permite modelar el comportamiento de robots en un entorno bidimensional. Para ejecutar las simulaciones de manera eficiente, especialmente en entornos como Jupyter, se utiliza la función **run_shell()**, que permite ejecutar BDSim desde un subshell, habilitando la visualización gráfica de las simulaciones en tiempo real. Este enfoque asegura que los resultados se puedan analizar en detalle, manteniendo la coherencia entre las entradas, simulaciones y resultados dentro del entorno de trabajo. En las simulaciones de este capítulo, se realizarán ejemplos, como la creación de mapas como los basados en cuadrículas y en hojas de ruta, que son fundamentales para entender las estrategias de control aplicables a los robots móviles.



```
if COLAB:
    !run -m braitenberg -H -g # no graphics
else:
    run_shell("braitenberg")
```

The screenshot shows a VS Code editor window with a Python script. The script has a conditional execution block: if COLAB is true, it runs '!run -m braitenberg -H -g # no graphics'; otherwise, it runs 'run_shell("braitenberg")'. The status bar at the bottom indicates the file is 'python' and the path is 'c:\Users\MateoAlmeida\Desktop\l0mo\robotic\localRep\roboticDev\lib\site-packages\RVC3\models\braitenberg.py'.

Figura 0. Implementación run_shell en PY con VSCODE

5.2 Introducción a la navegación basada en mapas.

El objetivo principal de este enfoque es determinar el **mejor camino** entre dos ubicaciones en función de un criterio definido, como la distancia más corta o el menor costo. Sin embargo, "mejor" puede incluir otros factores, como:

- **Facilidad de tránsito:** Puede ser preferible recorrer una distancia más larga si el terreno o camino es más fácil de atravesar.
- **Restricciones del vehículo:** Un planificador más avanzado puede considerar el tamaño, la cinemática y la dinámica del robot, evitando giros más cerrados de lo que puede ejecutar.

Esta sección conecta con la definición fundamental de un robot como una máquina orientada a objetivos que **percibe, planifica y actúa**. Aquí, la planificación se enfoca específicamente en la navegación, lo que implica el uso de mapas para resolver problemas de desplazamiento.

Tipos de representaciones de mapas

1. Mapas basados en cuadrículas (grids):

- **Definición:** Representan el entorno como una cuadrícula de celdas, generalmente cuadradas.
- **Tipos de celdas:**
 - **Ocupadas:** Indican obstáculos que el robot no puede atravesar.
 - **Desocupadas:** Representan áreas transitables.
- **Tamaño de las celdas:** Depende de la aplicación. Las celdas más pequeñas proporcionan mayor precisión, pero requieren más memoria. Sin embargo, la representación en cuadrículas es factible incluso para áreas grandes gracias a las capacidades de los computadores modernos.

Los grafos y las cuadrículas permiten a los robots planificar rutas considerando diversos factores. Por ejemplo:

- En grafos embebidos, un robot puede calcular la ruta más corta entre nodos.
- En grafos topológicos, un robot puede navegar reconociendo lugares y las conexiones entre ellos.
- En cuadrículas, un robot utiliza la ocupación de las celdas para evitar obstáculos y encontrar caminos transitables.

La navegación basada en mapas combina modelos matemáticos como cuadrículas para permitir que los robots planifiquen rutas óptimas considerando múltiples restricciones y factores del entorno. Esta capacidad es esencial para lograr un comportamiento autónomo eficiente, aunque se enfrentan desafíos como la representación precisa del entorno y la complejidad de los cálculos en áreas grandes o con múltiples restricciones.

5.4 Planificación con un mapa de cuadrícula de ocupación

La planificación de rutas en mapas basados en celdas de ocupación se utiliza para guiar a los robots a través de un entorno representado en una cuadrícula.

Cada celda de la cuadrícula puede estar ocupada (con un obstáculo) o libre. El objetivo es planificar la mejor ruta evitando los obstáculos.

Creación del Mapa de Ocupación

El primer paso es crear un mapa de ocupación donde las celdas pueden ser libres o ocupadas. A continuación, un ejemplo de código para generar un mapa de ocupación con obstáculos:

```
[3] map = np.zeros((100, 100));
    ✓ 0.0s

[4] map[40:50, 20:80] = 1; # set to occupied
    ✓ 0.0s

▷ [5] map.ravel()[np.random.choice(map.size, 100, replace=False)] = 1;
    ✓ 0.0s

▷ [10] simplegrid = np.zeros((6, 6));
      simplegrid[2:5, 3:5] = 1
      simplegrid[3:5, 2] = 1
    ✓ 0.0s
```

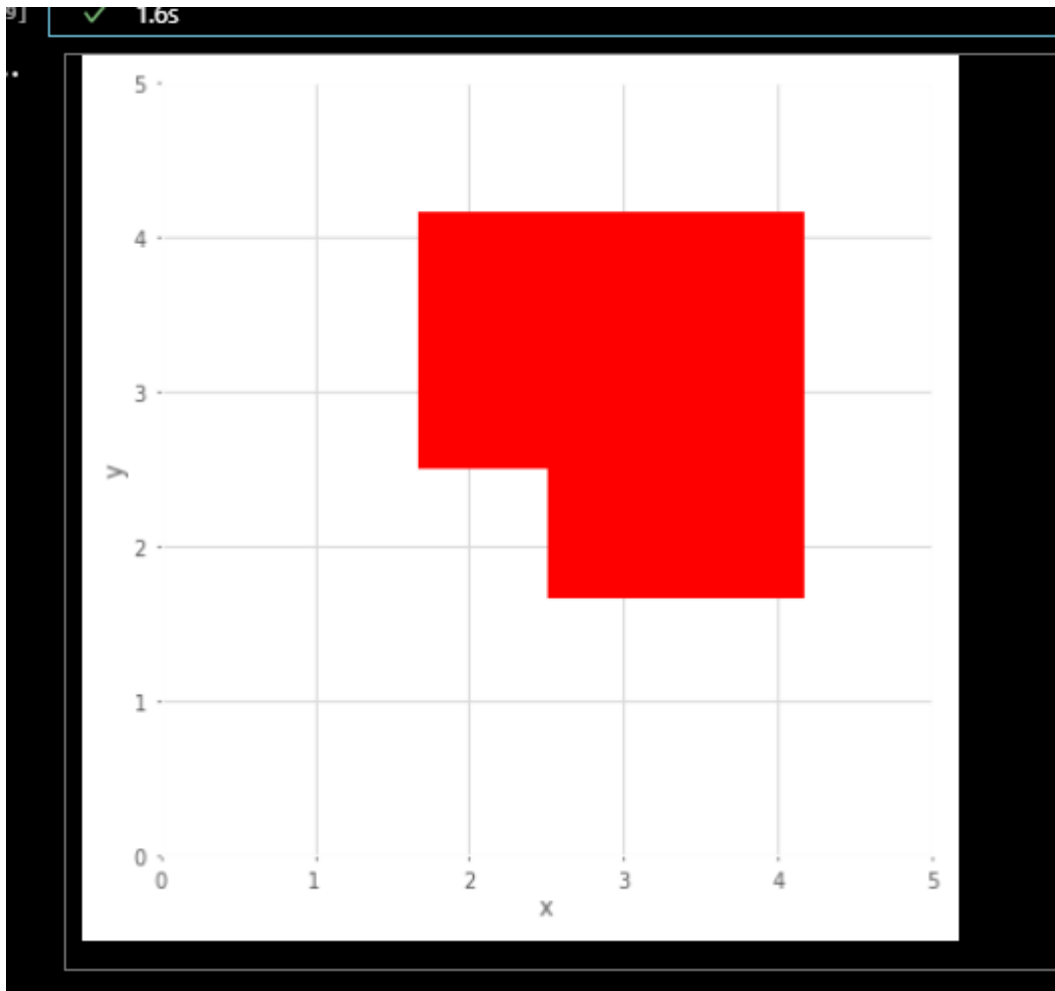


Figura 1. Mapa de ocupación en Python

Transformación de Distancia

Una vez creado el mapa de ocupación, la siguiente tarea es calcular la distancia de cada celda al objetivo utilizando una transformación de distancia. Esto permite planificar el camino evitando obstáculos.

El **método de transformación de distancia** calcula la distancia entre las celdas ocupadas y el objetivo para crear un mapa de planificación basado en la distancia. El proceso implica calcular la distancia de cada celda a la meta, permitiendo planificar la ruta más corta sin obstáculos.

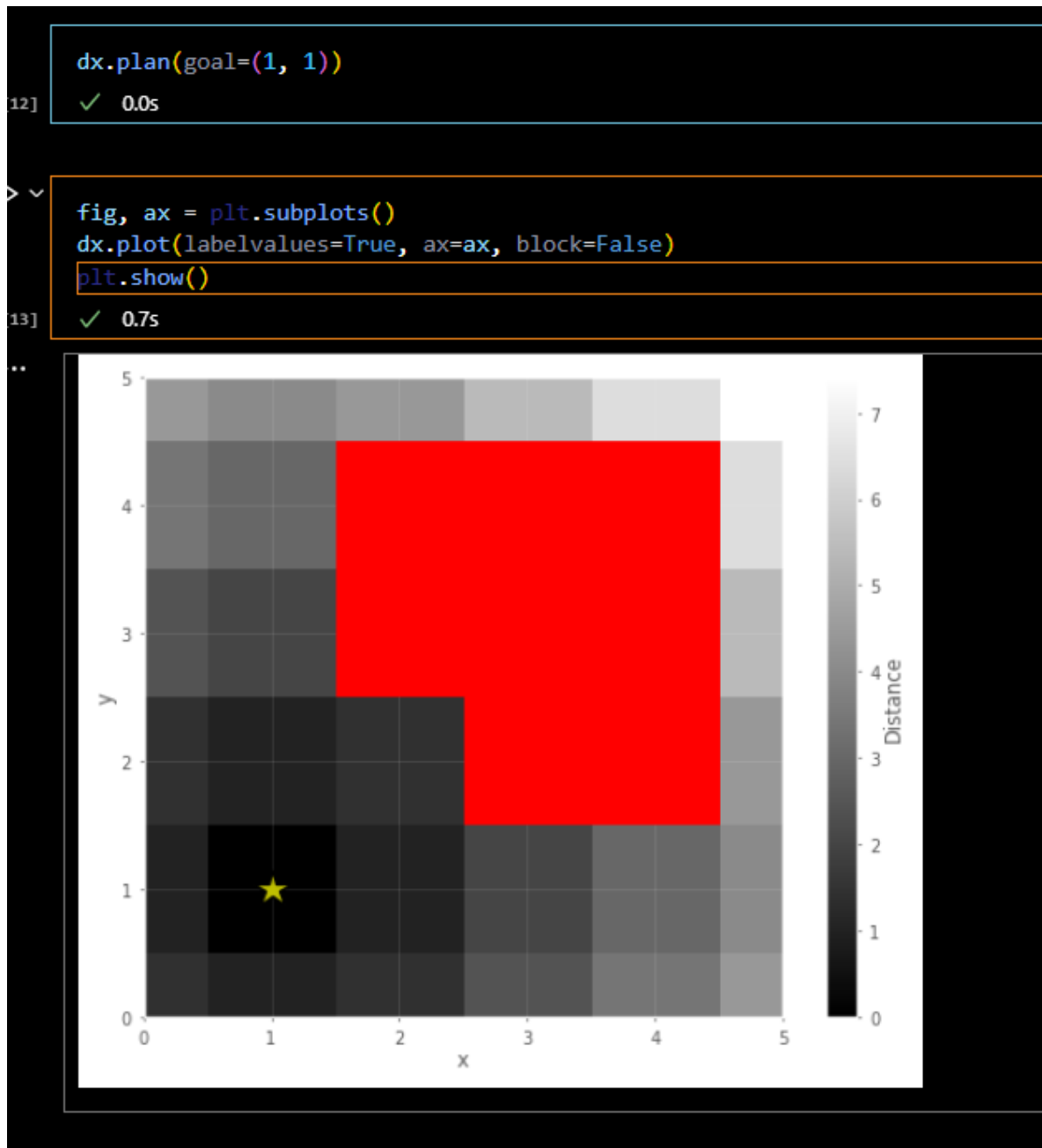


Figura 1.1 Resultado de la transformación de distancia, mostrando el mapa con la ruta hacia el objetivo.

El código utiliza un planificador de transformación de distancia para calcular la distancia entre cada celda y el objetivo, lo que permite definir el camino óptimo.

Transformación de distancia en 3D

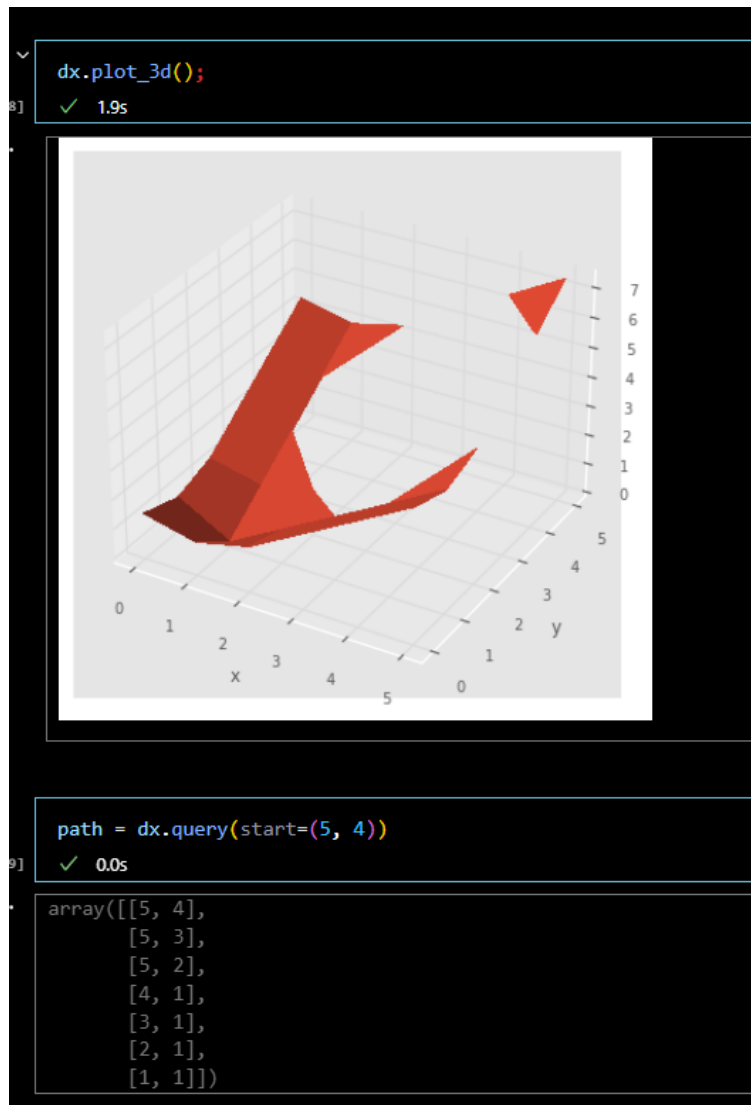


Figura 1.2 Transformación de distancia 3D

La imagen que se presenta muestra cómo se calcula la transformación de distancia utilizando un mapa de ocupación. En este caso, la función `dx.plot_3d()` permite visualizar la distancia desde el objetivo para cada celda del mapa.

El gráfico generado muestra el campo de distancias desde el objetivo. El robot debe moverse en el espacio y, dependiendo de los obstáculos, las distancias aumentan o disminuyen. La visualización en 3D proporciona una forma intuitiva de observar las áreas más cercanas al objetivo y las zonas más alejadas.

Este código y sus resultados reflejan cómo el mapa de ocupación con la transformación de distancia puede ser utilizado para guiar a un robot en una planificación de ruta eficiente, teniendo en cuenta los obstáculos y utilizando la distancia de Manhattan.

Código para planificación con mapas de ocupación

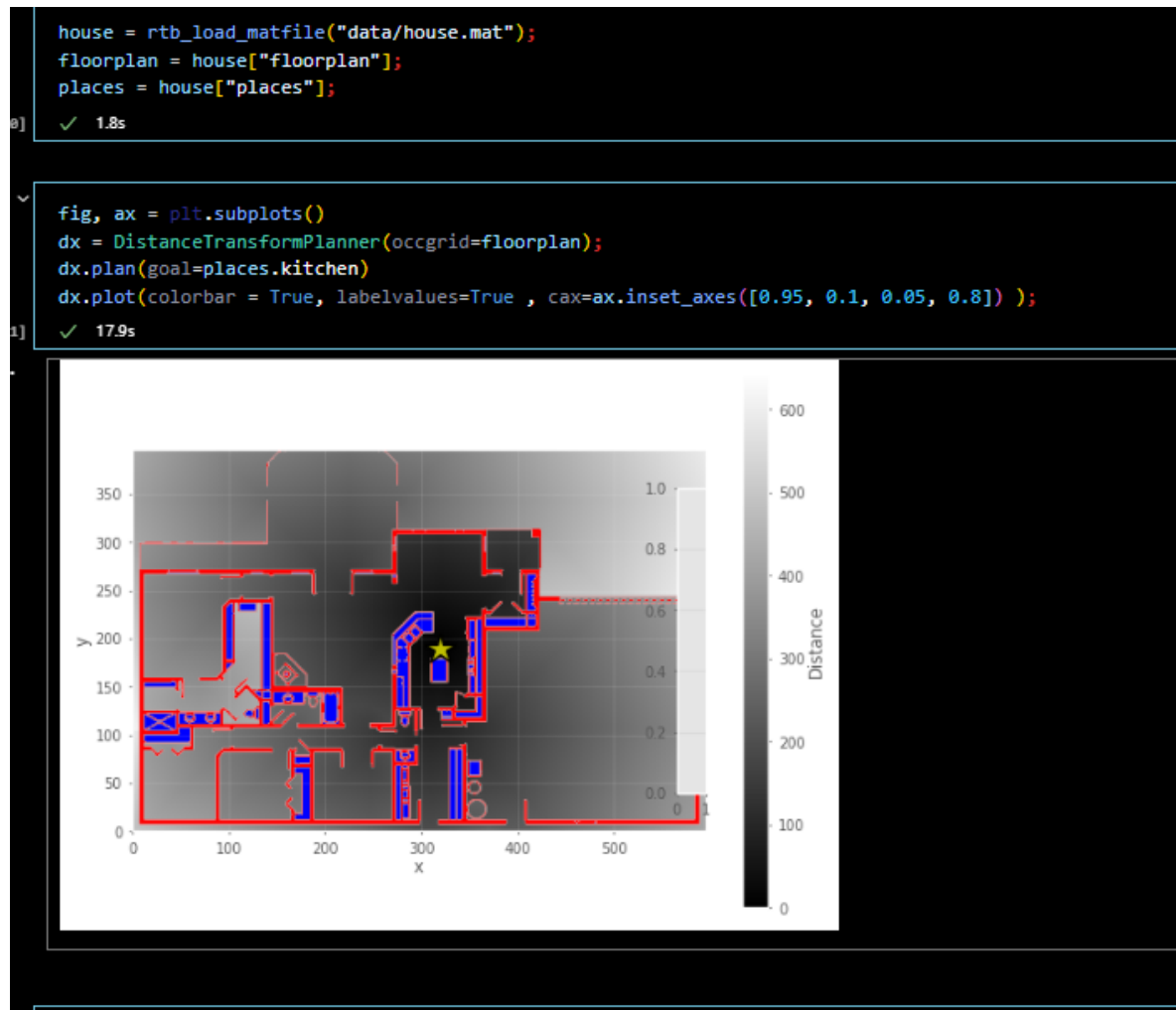


Figura 2 Planificación con mapas de ocupación en una casa

Carga del mapa: Utiliza un archivo `house.mat` que contiene la distribución de la casa y los lugares dentro de ella. La cuadrícula de ocupación (`floorplan`) es extraída de estos datos.

Planificación de la ruta: La función `dx.plan(goal=places["kitchen"])` calcula la ruta desde el punto inicial hacia el objetivo (la cocina), evitando obstáculos.

Visualización del mapa de ocupación: Utiliza `dx.plot()` para mostrar la cuadrícula con la ruta planificada, y un inset de color para representar las distancias.

Simulación del movimiento en robot (ruta hecha)

Una vez calculado el camino, el siguiente paso es simular el movimiento del robot desde su posición inicial hasta el objetivo (la cocina). A continuación, se muestra cómo realizar esta simulación y cómo obtener las coordenadas de la trayectoria:

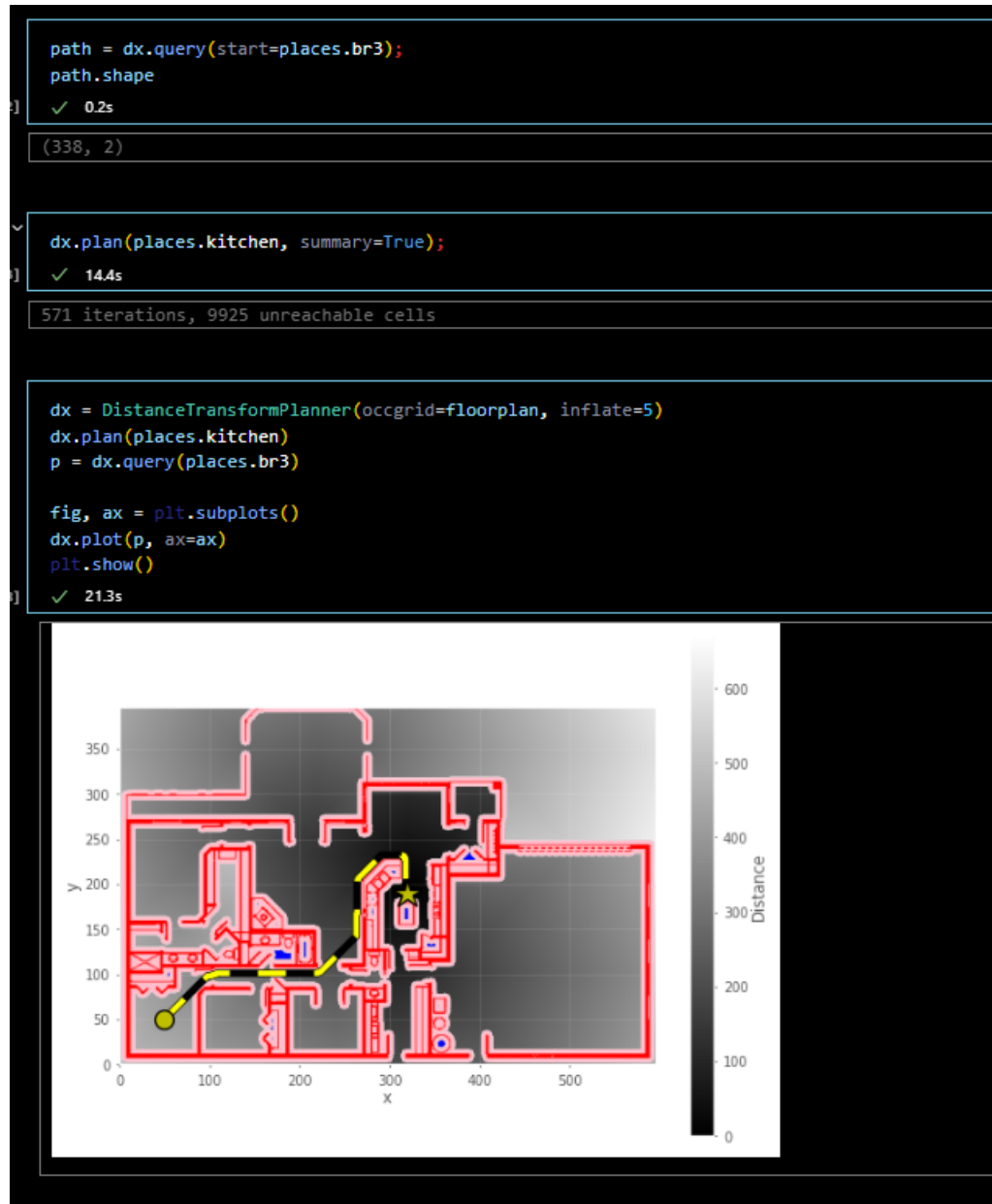


Figura 2.1 Planificación del robot desde su inicio hasta el objetivo (cocina), mostrando la ruta seguida en el mapa de ocupación.

Visualización con Inflado del Mapa de Ocupación

El siguiente paso es modificar el mapa de ocupación para tener en cuenta áreas adicionales alrededor del robot, representando un área de "seguridad" alrededor del robot. Esto se puede lograr mediante el parámetro `inflate`, que expande las áreas ocupadas en el mapa.



Figura 2.2 Mapa de ocupación con el área inflada, mostrando el robot y su camino desde el inicio hasta el objetivo.

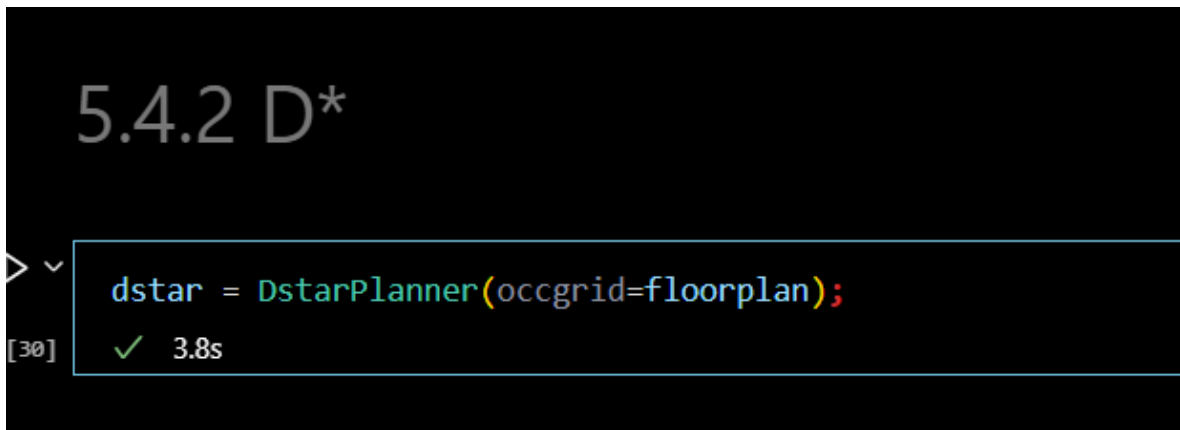
Vemos como la planificación en mapas de ocupación permite a los robots moverse de manera eficiente en entornos con obstáculos. A través de la transformación de distancia, se calculan las rutas óptimas hacia los objetivos, evitando áreas ocupadas. La visualización y simulación del movimiento del robot nos proporcionan una forma clara de entender cómo los robots utilizan estos mapas para navegar de manera segura.

1. Algoritmo D*:

En este apartado, se implementa el algoritmo **D***, que es una extensión dinámica del algoritmo **A*** y está diseñado para entornos donde los obstáculos pueden cambiar mientras el robot se desplaza. Este algoritmo permite realizar una planificación de ruta que se ajusta a medida que se descubren nuevos obstáculos, lo que es esencial en escenarios de navegación autónoma en entornos dinámicos. A continuación, se explica detalladamente cómo se implementa y cómo funciona el código utilizado para esta simulación.

Inicialización del Planificador D*

El primer paso es crear un objeto del planificador **D***, que utilizará un mapa de ocupación del entorno. El mapa de ocupación, en este caso, es un **floorplan** que contiene la representación del entorno en términos de celdas ocupadas y libres.



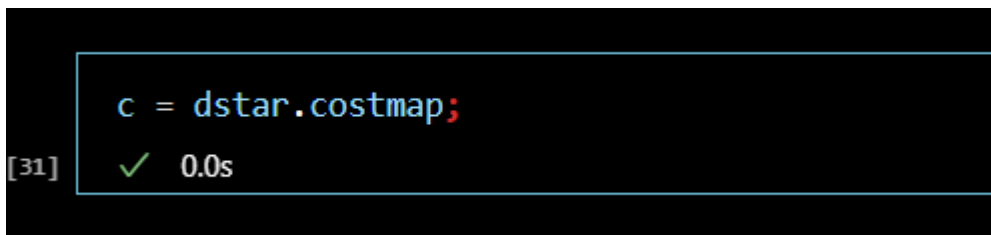
```
5.4.2 D*
> ✓ 3.8s
[30] dstar = DstarPlanner(occgrid=floorplan);
```

Figura 3. Algoritmo D*

Este código crea un planificador de rutas utilizando el mapa de ocupación cargado previamente en el entorno. **DstarPlanner** es el objeto que permite calcular las rutas, tomando en cuenta los obstáculos que puedan ir surgiendo durante la misión del robot.

Conversión del Mapa a un Mapa de Costos

Una de las características del algoritmo **D*** es que convierte el mapa de ocupación en un mapa de costos, donde las celdas ocupadas tienen un costo infinito (inaccesibles) y las celdas libres tienen costos bajos, dependiendo de la distancia a la meta.



```
[31] ✓ 0.0s
c = dstar.costmap;
```

Figura 3.1 Mapa de costos

Este paso es crucial porque, antes de planificar la ruta, el algoritmo necesita un mapa de costos que le permita determinar qué áreas son accesibles y qué áreas deben ser evitadas debido a obstáculos.

Planificación de la Ruta

A continuación, el algoritmo utiliza el **método plan** para generar la ruta desde un punto inicial hasta el objetivo (en este caso, la cocina). El planificador analiza el mapa de costos y calcula la mejor ruta posible considerando los obstáculos presentes.

```
[32] dstar.plan(goal=places.kitchen);  
✓ 1m 28.2s
```

Figura 3.2 Planificación de ruta

Este paso genera una planificación inicial de la ruta, partiendo de la posición inicial del robot hacia el objetivo, utilizando el algoritmo **D*** para identificar la trayectoria más eficiente teniendo en cuenta las restricciones del entorno.

Consulta de la Ruta Generada

Una vez que la planificación inicial está completada, podemos consultar la ruta generada para ver el camino específico que el robot debe seguir. Este paso es importante para obtener la lista de celdas o puntos que conforman el trayecto desde el punto de inicio hasta el objetivo.

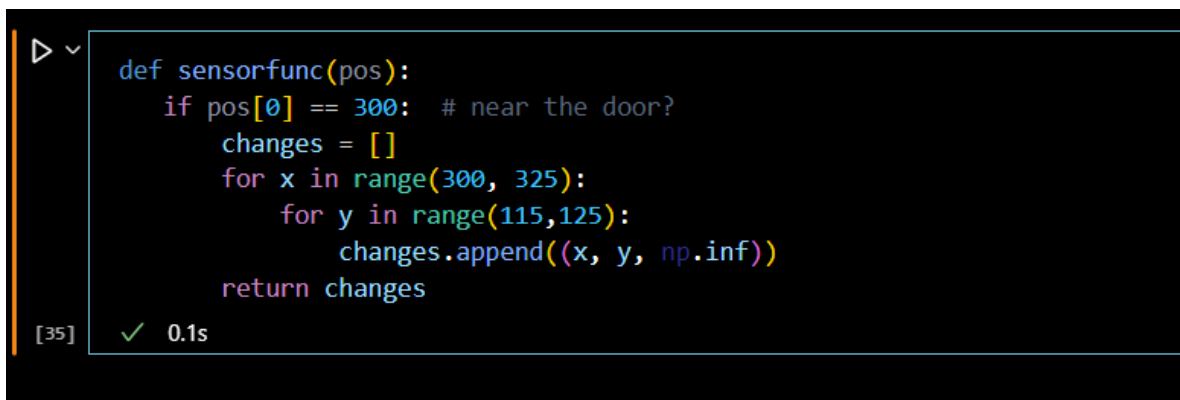
```
▷ ▾ nexpand0 = dstar.nexpand  
[33] ✓ 0.0s  
... 359764  
  
▷ ▾ path = dstar.query(start=places.br3);  
[34] ✓ 0.2s
```

Figura 3.3 Ruta generada

El código obtiene la ruta que el robot debe seguir desde la posición de inicio `places["br3"]` hasta la cocina. La función `query` genera la ruta consultando el mapa de costos y considerando cualquier obstáculo presente.

Función de Sensor para el Cambio del Mapa

Un aspecto clave del algoritmo **D*** es su capacidad para adaptarse a los cambios en el entorno durante la ejecución. Esto se logra mediante el uso de una función de sensor que detecta los cambios en el mapa, como nuevos obstáculos que aparecen durante el trayecto del robot.

A screenshot of a code editor with a dark background. On the left, there is a vertical orange bar and a small icon of a play button with a checkmark. The code is written in a light blue font. It defines a function `sensorfunc(pos):`. Inside the function, there is an `if` statement: `if pos[0] == 300: # near the door?`. If true, it initializes `changes = []`, then enters two nested `for` loops: `for x in range(300, 325):` and `for y in range(115, 125):`. Inside the inner loop, it appends a tuple `(x, y, np.inf)` to the `changes` list. Finally, it `return changes`. At the bottom left, there is a status bar showing `[35]`, a green checkmark, and `0.1s`.

```
def sensorfunc(pos):
    if pos[0] == 300: # near the door?
        changes = []
        for x in range(300, 325):
            for y in range(115, 125):
                changes.append((x, y, np.inf))
        return changes
```

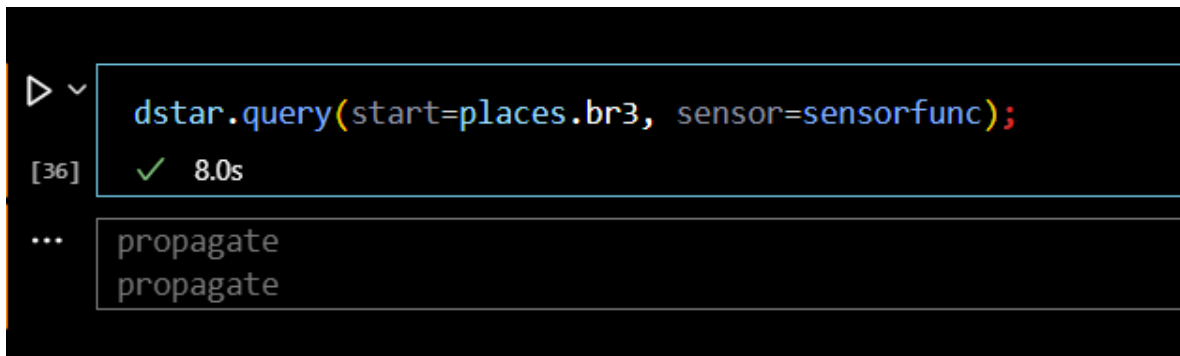
[35] ✓ 0.1s

Figura 3.4 Sensor para cambio de mapa

La función `sensorfunc` está diseñada para bloquear el camino cuando el robot detecta que está cerca de un obstáculo, como una puerta cerrada (en la posición `pos[0] == 300`). Esta función modifica el mapa de costos, asignando un valor infinito a las celdas cercanas, lo que indica que esas áreas se han vuelto inalcanzables.

Replanificación en Tiempo Real

Luego de aplicar la función del sensor, el algoritmo **D*** puede replanificar la ruta considerando los nuevos obstáculos descubiertos. La replanificación se realiza cada vez que el robot detecta un cambio en su entorno, asegurando que siempre siga el camino más seguro disponible.



```
dstar.query(start=places.br3, sensor=sensorfunc);
```

[36] ✓ 8.0s

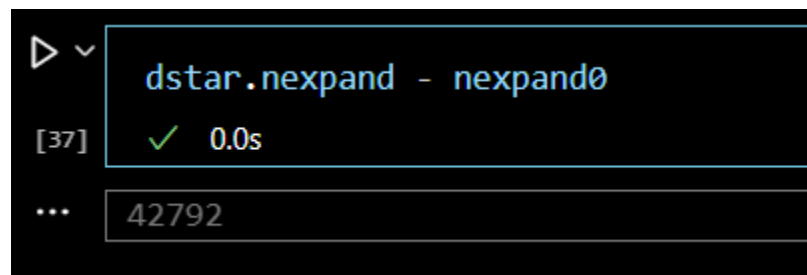
... propagate

Figura 3.5 Replanificación en tiempo real

Aquí, el robot consulta nuevamente el camino a seguir, tomando en cuenta la función del sensor que ha detectado cambios en el mapa y actualizando la ruta en tiempo real.

Cálculo de Expansión de Celdas

Una característica interesante de **D*** es que puede medir cuántas celdas fueron necesarias para calcular la ruta. Esto ayuda a evaluar la eficiencia del algoritmo en términos de computación y memoria.



```
dstar.nexpand - nexpand0
```

[37] ✓ 0.0s

... 42792

Figura 3.6 Expansión de celdas

El número de celdas expandidas (es decir, cuántas celdas fueron revisadas para encontrar la ruta más corta) se calcula comparando el valor actual con el valor inicial. Este valor es útil para evaluar el rendimiento del algoritmo en diferentes condiciones.

5.5 Planificación con hojas de ruta.

En esta sección se discuten las hojas de ruta en el contexto de la planificación de trayectorias para robots. A diferencia de la planificación basada en mapas de ocupación, las hojas de ruta implican la creación de una red de caminos (red de vértices y aristas) en el espacio libre, que sirve para conectar puntos en un espacio.

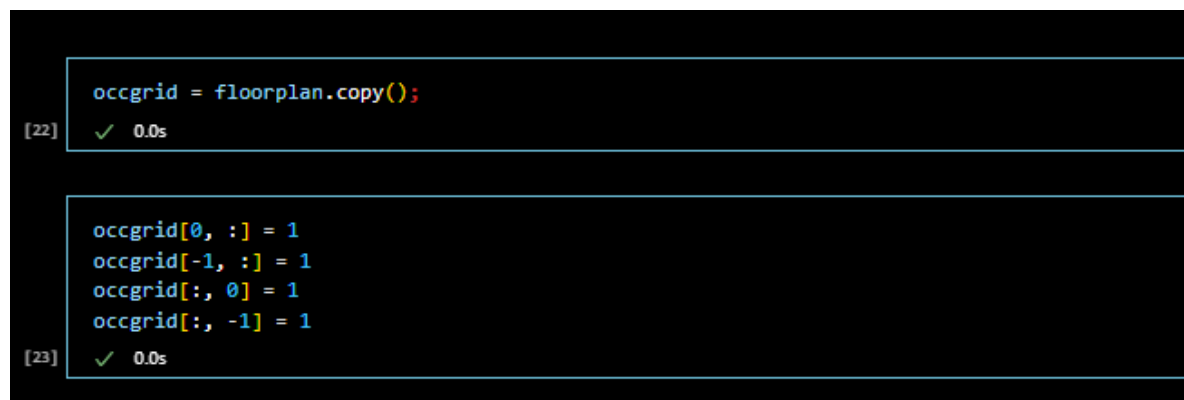
La idea básica de la planificación con hojas de ruta es construir una red de caminos que represente el espacio libre. Cada punto o lugar del espacio es un vértice del grafo, y los caminos entre los vértices son las aristas del grafo. Estas hojas de ruta pueden usarse para facilitar la planificación de trayectorias evitando obstáculos y reduciendo la complejidad computacional al no requerir la planificación de cada punto de forma individual.

En el caso de los robots móviles, se pueden generar hojas de ruta en entornos de mapas de ocupación, y en este caso se utiliza la estrategia de **Probabilistic Roadmap** (PRM), que permite crear conexiones entre puntos al azar dentro de un entorno.

Veamos cómo procedemos usando las herramientas de Toolbox para robótica:

1. Creación del Mapa de Ocupación:

El primer paso es crear una copia del mapa de ocupación que se utilizará para la planificación de la trayectoria, utilizando el código siguiente:



```
[22] ✓ 0.0s
occgrid = floorplan.copy();

[23] ✓ 0.0s
occgrid[0, :] = 1
occgrid[-1, :] = 1
occgrid[:, 0] = 1
occgrid[:, -1] = 1
```

Figura 4. Creación mapa de ocupación

Aquí, estamos creando un mapa de ocupación `occgrid` a partir del mapa de planta del entorno (`floorplan`). Las líneas de código posteriores aseguran que los bordes del mapa estén marcados como ocupados, lo que ayuda a evitar que el robot pase por los límites del entorno.

2. Visualización del Mapa de Ocupación:

Luego, se visualiza el mapa de ocupación utilizando la herramienta de la biblioteca *Machine Vision Toolbox*:



Figura 4.1 Visualización del mapa de ocupación

Aquí, se crea un objeto de imagen de la librería, que tiene en cuenta las celdas vacías ($\text{occgrid} == 0$) del mapa de ocupación. Luego, la imagen se muestra utilizando `disp()`.

3. Esqueleto del Mapa:

Para crear un esqueleto del mapa, se utiliza una transformación para hacer la simplificación del espacio libre y obtener una representación simplificada de las áreas transitables. Esto se hace con el siguiente código:

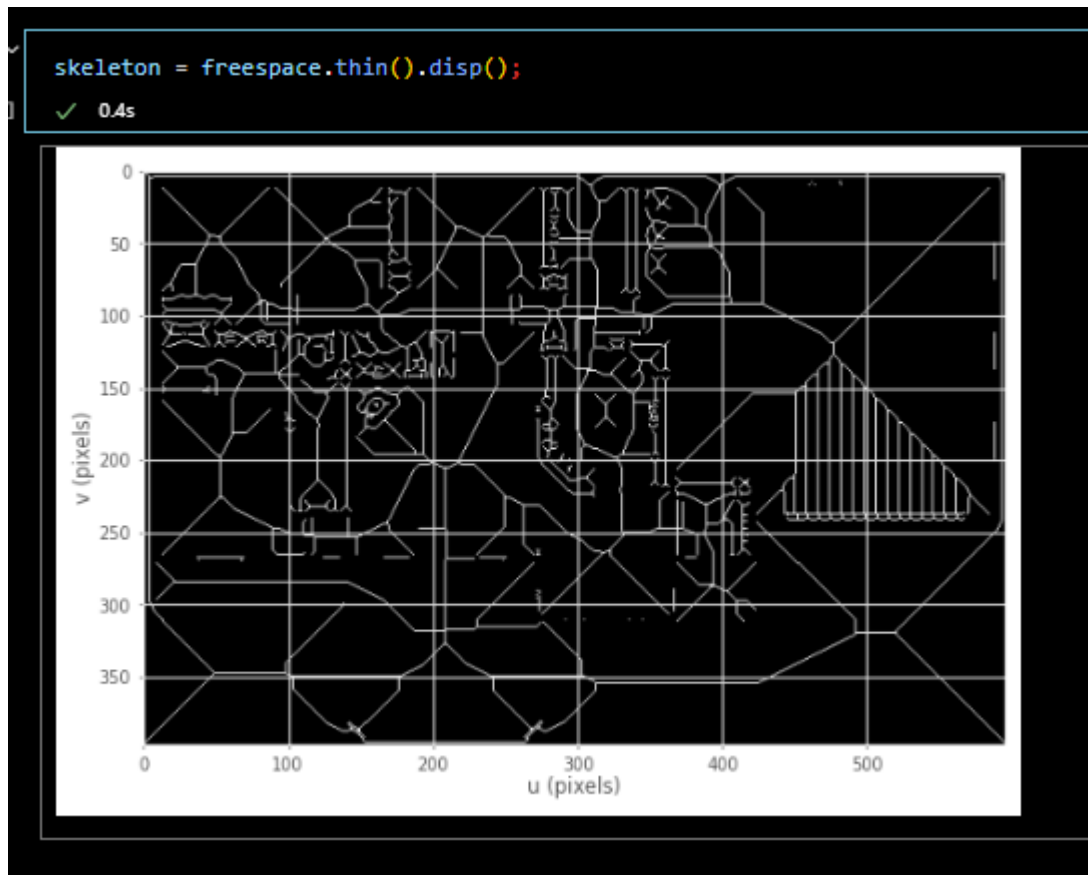


Figura 4.2 Esqueleto del mapa

Este comando genera un "esqueleto" del espacio libre del mapa, reduciendo las áreas ocupadas y resaltando solo las rutas por donde el robot puede pasar. La visualización del esqueleto ayuda a comprender mejor las áreas transitables y las zonas bloqueadas por obstáculos.

4. Generación de Puntos Aleatorios y Roadmap:

A continuación, se generan puntos aleatorios que estarán conectados entre sí para formar el grafo de la hoja de ruta. Este paso es crucial para crear la red de caminos del PRM.

Excuse 5.8: Voronoi Tessellation

```
> ~  
sites = np.random.uniform(size=(2,9));  
from scipy.spatial import Voronoi  
  
vor = Voronoi(sites.T);  
  
from scipy.spatial import voronoi_plot_2d
```

Figura 4.3 Voronoi tessellation

Los puntos generados aleatoriamente forman el conjunto de sitios sobre los cuales se construye la **Diagrama de Voronoi**. Cada sitio o punto en el diagrama es una posible ubicación para el robot.

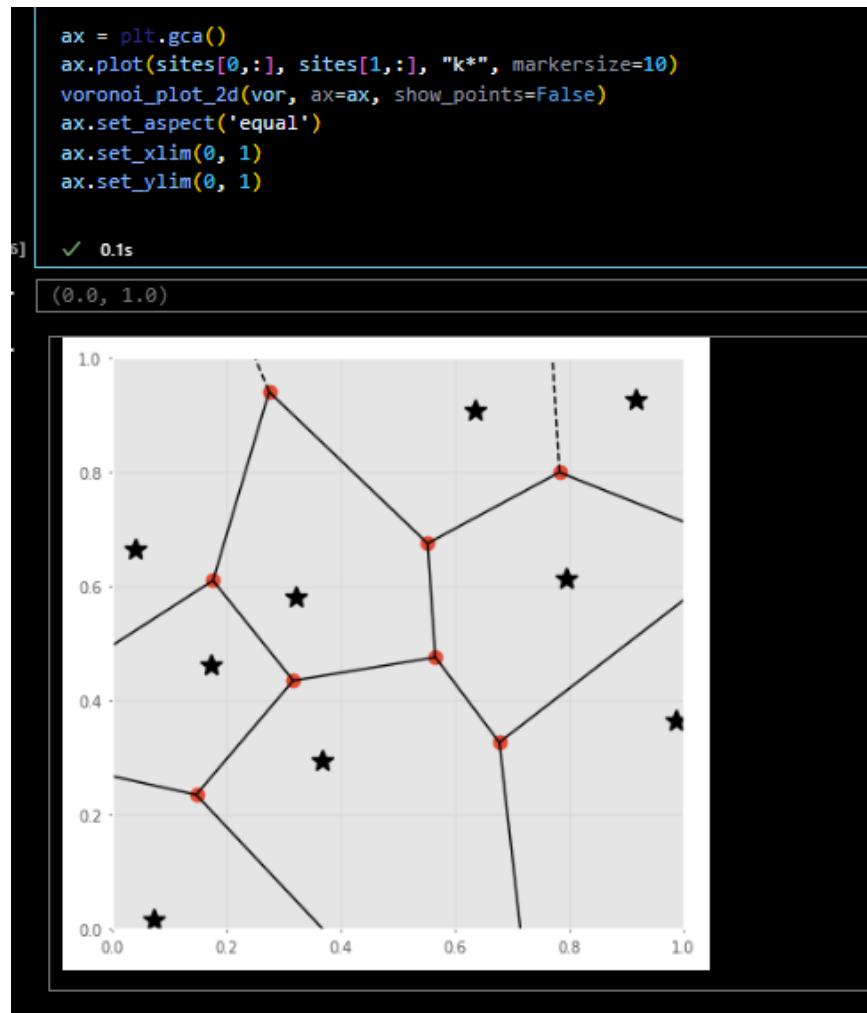


Figura 4.4 Diagrama de voronoi

Con este código, visualizamos el Diagrama de Voronoi en un gráfico, donde los puntos se representan con asteriscos negros. Este diagrama es clave porque nos permite entender cómo se distribuyen los sitios en el espacio libre.

5. Generación del PRM (Probabilistic Roadmap):

El siguiente paso es crear el planificador PRM para conectar los puntos aleatorios y generar la hoja de ruta:

```
prm = PRMPlanner(occgrid=floorplan, seed=0);  
✓ 0.0s  
  
prm.plan(npoints=50)  
✓ 0.3s  
  
prm  
✓ 0.0s  
  
PRMPlanner:  
  BinaryOccupancyGrid: 596 x 397, cell size=1, x = [0.0, 595.0], y = [0.0, 396.0], 8.8% occupied  
  UGraph: 50 vertices, 222 edges, 12 components
```

Figura 4.5 Generación del PRM

Este código genera un planificador de hoja de ruta probabilística (*PRMPlanner*), que usa los puntos generados previamente en el espacio de ocupación. El parámetro *npoints* indica el número de puntos que se deben generar para formar la red de caminos.

6. Visualización del Roadmap:

Finalmente, el gráfico de la hoja de ruta generada se visualiza con los siguientes comandos:



Figura 4.6 Visualización de la hoja de ruta con 30 iteraciones.

Figura 5.17: Muestra un gráfico con el mapa de ocupación modificado dinámicamente. Las áreas de alto costo son indicadas en rojo y amarillo, mientras que las rutas de baja costo son representadas en azul.

Esta visualización nos muestra el grafo generado con los 50 puntos en el mapa de ocupación, y las conexiones entre ellos. Los puntos representarán las ubicaciones posibles en el espacio libre, y las líneas representarán los caminos que el robot puede tomar para navegar entre esos puntos.

De igual forma, podemos iterar de manera precisa para observar los cambios tanto en la generación de puntos aleatorios, al cubrir mayor parte del mapa de ocupación y también en la ruta a seguir por el robot para llegar al objetivo definido.



Figura 4.7 Visualización de la hoja de ruta con 300 iteraciones

Haciendo uso del parámetro query, podemos proceder a trazar la ruta.

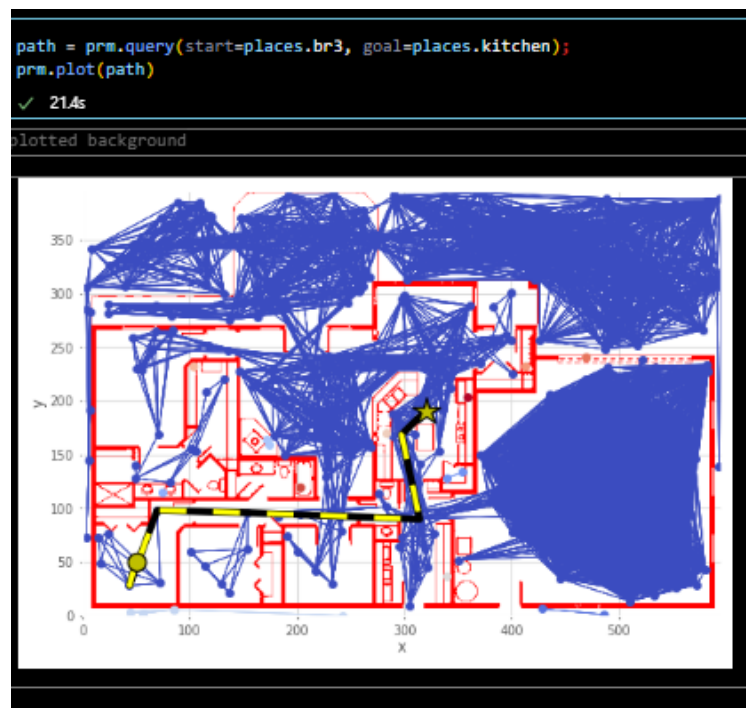


Figura 4.8 Ruta del robot una vez iterado 300 veces.

El uso de **Planificación con Hojas de Ruta (PRM)** es fundamental para reducir la complejidad de la planificación de trayectorias en entornos grandes y complejos. A través de la generación de puntos aleatorios y la creación de un grafo de conectividad, el robot puede navegar a través del espacio sin necesidad de calcular caminos para cada punto individualmente. Esto permite una planificación eficiente y adaptable a entornos dinámicos, como se demuestra en los ejemplos anteriores.

Conclusiones

- Las simulaciones realizadas mostraron que los **robots con navegación basada en mapas de ocupación** pueden alcanzar objetivos de forma eficiente sin necesidad de planificación previa. Estos robots pueden detectar y evitar obstáculos mientras siguen una trayectoria hacia el objetivo, utilizando información de los sensores para ajustar su ruta en tiempo real. Esta capacidad es esencial para operar en entornos complejos y dinámicos, donde los obstáculos pueden cambiar repentinamente.
- El algoritmo utilizado para **planificación con mapas de ocupación** demostró la efectividad de las representaciones de mapas, como el **mapa de ocupación binario** y la **transformación de distancia**, que permiten encontrar la ruta más corta libre de obstáculos. Las gráficas de las distancias y el comportamiento del robot a través del mapa confirmaron que los métodos basados en mapas de ocupación son capaces de planificar trayectorias eficientes y seguras en espacios mapeados.
- En cuanto a los **algoritmos de planificación con hojas de ruta** (como PRM), se demostró que son efectivos para generar rutas eficientes en un espacio de planificación libre de obstáculos. Al construir una **red de caminos** o **roadmap**, estos algoritmos permiten realizar simulaciones de navegación que optimizan las rutas en función de la disponibilidad de puntos intermedios entre el inicio y el objetivo. Las simulaciones mostraron cómo el **PRM** explora el espacio de manera eficiente, creando redes de conexiones entre los puntos del mapa que guían al robot a través de rutas óptimas.
- Las gráficas que muestran las trayectorias del robot en el mapa de ocupación y el roadmap validaron la efectividad de ambos enfoques de planificación, mostrando cómo el robot sigue las rutas generadas por los algoritmos mientras se adapta a los cambios en el entorno.

En resumen, tanto **la planificación con mapas de ocupación** como **la planificación con hojas de ruta** son herramientas poderosas para guiar robots autónomos en entornos

estructurados. Los mapas de ocupación permiten detectar y evitar obstáculos, mientras que las hojas de ruta proporcionan una planificación de trayectoria más global y eficiente. Combinados con algoritmos adecuados, estos enfoques son fundamentales para construir **sistemas de navegación autónoma robustos** que pueden operar en una variedad de entornos, tanto dinámicos como estáticos.

Referencias

- UNTRACE folder.
- Corke, P. Robotics, Vision and Control v2 (RVC)
- notebook : chap5.ipynb
- notebook : chap5CP.ipynb
- Documentación Robotics Toolbox
- Git hub RVC v2